What's new in Claude 4.5

Copy page

Claude 4.5 introduces three models designed for different use cases:

- **Claude Opus 4.5**: Our most intelligent model combining maximum capability with practical performance. Features a more accessible price point than previous Opus models. Available with a 200k token context window.
- **Claude Sonnet 4.5**: Our best model for complex agents and coding, with the highest intelligence across most tasks. Available with a 200k and 1M (beta) token context window.
- **Claude Haiku 4.5**: Our fastest and most intelligent Haiku model with near-frontier performance. Available with a 200k token context window.

Key improvements in Opus 4.5 over Opus 4.1

Maximum intelligence

Claude Opus 4.5 represents our most intelligent model, combining maximum capability with practical performance. It delivers step-change improvements across reasoning, coding, and complex problem-solving tasks while maintaining the high-quality outputs expected from the Opus family.

Effort parameter

Claude Opus 4.5 is the only model that supports the effort parameter, allowing you to control how many tokens Claude uses when responding. This gives you the ability to trade off between response thoroughness and token efficiency with a single model.

The effort parameter affects all tokens in the response, including text responses, tool calls, and extended thinking. You can choose between:

- **High effort**: Maximum thoroughness for complex analysis and detailed explanations
- **Medium effort**: Balanced approach for most production use cases
- **Low effort**: Most token-efficient responses for high-volume automation

Claude Opus 4.5 introduces enhanced computer use capabilities with a new zoom action that enables detailed inspection of specific screen regions at full resolution. This allows Claude to examine fine-grained UI elements, small text, and detailed visual information that might be unclear in standard screenshots.

The zoom capability is particularly valuable for:

- Inspecting small UI elements and controls
- Reading fine print or detailed text
- Analyzing complex interfaces with dense information
- Verifying precise visual details before taking actions

Practical performance

Claude Opus 4.5 delivers flagship intelligence at a more accessible price point than previous Opus models, making advanced AI capabilities available for a broader range of applications and use cases.

Thinking block preservation

Claude Opus 4.5 automatically preserves all previous thinking blocks throughout conversations, maintaining reasoning continuity across extended multi-turn interactions and tool use sessions. This ensures Claude can effectively leverage its full reasoning history when working on complex, long-running tasks.

Key improvements in Sonnet 4.5 over Sonnet 4

Coding excellence

Claude Sonnet 4.5 is our best coding model to date, with significant improvements across the entire development lifecycle:

- **SWE-bench Verified performance**: Advanced state-of-the-art on coding benchmarks
- **Enhanced planning and system design**: Better architectural decisions and code organization
- **Improved security engineering**: More robust security practices and vulnerability detection
- **Better instruction following**: More precise adherence to coding specifications and requirements

Claude Sonnet 4.5 performs significantly better on coding tasks when extended thinking is enabled. Extended thinking is disabled by default, but we recommend enabling it for complex coding work. Be aware

Agent capabilities

Claude Sonnet 4.5 introduces major advances in agent capabilities:

- **Extended autonomous operation**: Sonnet 4.5 can work independently for hours while maintaining clarity and focus on incremental progress. The model makes steady advances on a few tasks at a time rather than attempting everything at once. It provides fact-based progress updates that accurately reflect what has been accomplished.
- **Context awareness**: Claude now tracks its token usage throughout conversations, receiving updates after each tool call. This awareness helps prevent premature task abandonment and enables more effective execution on long-running tasks. See Context awareness for technical details and prompting guidance.
- **Enhanced tool usage**: The model more effectively uses parallel tool calls, firing off multiple speculative searches simultaneously during research and reading several files at once to build context faster. Improved coordination across multiple tools and information sources enables the model to effectively leverage a wide range of capabilities in agentic search and coding workflows.
- **Advanced context management**: Sonnet 4.5 maintains exceptional state tracking in external files, preserving goal-orientation across sessions. Combined with more effective context window usage and our new context management API features, the model optimally handles information across extended sessions to maintain coherence over time.

> Context awareness is available in Claude Sonnet 4, Sonnet 4.5, Haiku 4.5, Opus 4, Opus 4.1, and Opus 4.5.

Communication and interaction style

Claude Sonnet 4.5 has a refined communication approach that is concise, direct, and natural. It provides fact-based progress updates and may skip verbose summaries after tool calls to maintain workflow momentum (though this can be adjusted with prompting).

For detailed guidance on working with this communication style, see Claude 4 best practices.

Creative content generation

Claude Sonnet 4.5 excels at creative content tasks:

- **Presentations and animations**: Matches or exceeds Claude Opus 4.1 and Opus 4.5 for creating slides and visual content
- **Creative flair**: Produces polished, professional output with strong instruction following

Key improvements in Haiku 4.5 over Haiku 3.5

Claude Haiku 4.5 represents a transformative leap for the Haiku model family, bringing frontier capabilities to our fastest model class:

Near-frontier intelligence with blazing speed

Claude Haiku 4.5 delivers near-frontier performance matching Sonnet 4 at significantly lower cost and faster speed:

- **Near-frontier intelligence**: Matches Sonnet 4 performance across reasoning, coding, and complex tasks
- **Enhanced speed**: More than twice the speed of Sonnet 4, with optimizations for output tokens per second (OTPS)
- **Optimal cost-performance**: Near-frontier intelligence at one-third the cost, ideal for high-volume deployments

Extended thinking capabilities

Claude Haiku 4.5 is the **first Haiku model** to support extended thinking, bringing advanced reasoning capabilities to the Haiku family:

- **Reasoning at speed**: Access to Claude's internal reasoning process for complex problem-solving
- **Thinking Summarization**: Summarized thinking output for production-ready deployments
- **Interleaved thinking**: Think between tool calls for more sophisticated multi-step workflows
- **Budget control**: Configure thinking token budgets to balance reasoning depth with speed

Extended thinking must be enabled explicitly by adding a `thinking` parameter to your API requests. See the Extended thinking documentation for implementation details.

> Claude Haiku 4.5 performs significantly better on coding and reasoning tasks when extended thinking is enabled. Extended thinking is disabled by default, but we recommend enabling it for complex problem-solving, coding work, and multi-step reasoning. Be aware that extended thinking impacts prompt caching efficiency. See the migration guide for configuration details.

> Available in Claude Sonnet 3.7, Sonnet 4, Sonnet 4.5, Haiku 4.5, Opus 4, Opus 4.1, and Opus 4.5.

Context awareness

- **Token budget tracking**: Claude receives real-time updates on remaining context capacity after each tool call
- **Better task persistence**: The model can execute tasks more effectively by understanding available working space
- **Multi-context-window workflows**: Improved handling of state transitions across extended sessions

This is the first Haiku model with native context awareness capabilities. For prompting guidance, see Claude 4 best practices.

Available in Claude Sonnet 4, Sonnet 4.5, Haiku 4.5, Opus 4, Opus 4.1, and Opus 4.5.

Strong coding and tool use

Claude Haiku 4.5 delivers robust coding capabilities expected from modern Claude models:

- **Coding proficiency**: Strong performance across code generation, debugging, and refactoring tasks
- **Full tool support**: Compatible with all Claude 4 tools including bash, code execution, text editor, web search, and computer use
- **Enhanced computer use**: Optimized for autonomous desktop interaction and browser automation workflows
- **Parallel tool execution**: Efficient coordination across multiple tools for complex workflows

Haiku 4.5 is designed for use cases that demand both intelligence and efficiency:

- **Real-time applications**: Fast response times for interactive user experiences
- **High-volume processing**: Cost-effective intelligence for large-scale deployments
- **Free tier implementations**: Premium model quality at accessible pricing
- **Sub-agent architectures**: Fast, intelligent agents for multi-agent systems
- **Computer use at scale**: Cost-effective autonomous desktop and browser automation

New API features

Programmatic tool calling (Beta)

This significantly reduces latency for multi-tool workflows and decreases token consumption by allowing Claude to filter or process data before it reaches the model's context window.

```
tools=[
    {
        "type": "code_execution_20250825",
        "name": "code_execution"
    },
    {
        "name": "query_database",
        "description": "Execute a SQL query against the sales database. Returns a list of
        "input_schema": {...},
        "allowed_callers": ["code_execution_20250825"]  # Enable programmatic calling
    }
]
```

Key benefits:

- **Reduced latency**: Eliminate model round-trips between tool calls
- **Token efficiency**: Process and filter tool results programmatically before returning to Claude
- **Complex workflows**: Support loops, conditional logic, and batch processing

> Available in Claude Opus 4.5 and Claude Sonnet 4.5. Requires beta header: `advanced-tool-use-2025-11-20`

Tool search tool (Beta)

The tool search tool enables Claude to work with hundreds or thousands of tools by dynamically discovering and loading them on-demand. Instead of loading all tool definitions into the context window upfront, Claude searches your tool catalog and loads only the tools it needs.

Two search variants are available:

- **Regex** (`tool_search_tool_regex_20251119`): Claude constructs regex patterns to search tool names, descriptions, and arguments
- **BM25** (`tool_search_tool_bm25_20251119`): Claude uses natural language queries to search for tools

```
        "type": "tool_search_tool_regex_20251119",
        "name": "tool_search_tool_regex"
    },
    {
        "name": "get_weather",
        "description": "Get the weather at a specific location",
        "input_schema": {...},
        "defer_loading": True  # Load on-demand via search
    }
]
```

This approach solves two critical challenges:

- **Context efficiency**: Save 10-20K tokens by not loading all tool definitions upfront
- **Tool selection accuracy**: Maintain high accuracy even with 100+ available tools

> Available in Claude Opus 4.5 and Claude Sonnet 4.5. Requires beta header: `advanced-tool-use-2025-11-20`

Effort parameter (Beta)

The effort parameter allows you to control how many tokens Claude uses when responding, trading off between response thoroughness and token efficiency:

```
response = client.beta.messages.create(
    model="claude-opus-4-5-20251101",
    betas=["effort-2025-11-24"],
    max_tokens=4096,
    messages=[{"role": "user", "content": "..."}],
    output_config={
        "effort": "medium"  # "low", "medium", or "high"
    }
)
```

The effort parameter affects all tokens in the response, including text responses, tool calls, and extended thinking. Lower effort levels produce more concise responses with minimal explanations, while higher effort provides detailed reasoning and comprehensive answers.

> Available exclusively in Claude Opus 4.5. Requires beta header: `effort-2025-11-24`

Tool use examples allow you to provide concrete examples of valid tool inputs to help Claude understand how to use your tools more effectively. This is particularly useful for complex tools with nested objects, optional parameters, or format-sensitive inputs.

```
tools=[
    {
        "name": "get_weather",
        "description": "Get the current weather in a given location",
        "input_schema": {...},
        "input_examples": [
            {
                "location": "San Francisco, CA",
                "unit": "fahrenheit"
            },
            {
                "location": "Tokyo, Japan",
                "unit": "celsius"
            },
            {
                "location": "New York, NY"  # Demonstrates optional 'unit' parameter
            }
        ]
    }
]
```

Examples are included in the prompt alongside your tool schema, showing Claude concrete patterns for well-formed tool calls. Each example must be valid according to the tool's `input_schema`.

> Available in Claude Sonnet 4.5, Haiku 4.5, Opus 4.5, Opus 4.1, and Opus 4. Requires beta header: `advanced-tool-use-2025-11-20`.

Memory tool (Beta)

The new memory tool enables Claude to store and retrieve information outside the context window:

```
tools=[
    {
        "type": "memory_20250818",
        "name": "memory"
    }
]
```

- Building knowledge bases over time
- Maintaining project state across sessions
- Preserving effectively unlimited context through file-based storage

> ⓘ Available in Claude Sonnet 4, Sonnet 4.5, Haiku 4.5, Opus 4, Opus 4.1, and Opus 4.5. Requires beta header: `context-management-2025-06-27`

Context editing

Use context editing for intelligent context management through automatic tool call clearing:

```
response = client.beta.messages.create(
    betas=["context-management-2025-06-27"],
    model="claude-sonnet-4-5",  # or claude-haiku-4-5
    max_tokens=4096,
    messages=[{"role": "user", "content": "..."}],
    context_management={
        "edits": [
            {
                "type": "clear_tool_uses_20250919",
                "trigger": {"type": "input_tokens", "value": 500},
                "keep": {"type": "tool_uses", "value": 2},
                "clear_at_least": {"type": "input_tokens", "value": 100}
            }
        ]
    },
    tools=[...]
)
```

This feature automatically removes older tool calls and results when approaching token limits, helping manage context in long-running agent sessions.

> ⓘ Available in Claude Sonnet 4, Sonnet 4.5, Haiku 4.5, Opus 4, Opus 4.1, and Opus 4.5. Requires beta header: `context-management-2025-06-27`

Enhanced stop reasons

Claude 4.5 models introduce a new `model_context_window_exceeded` stop reason that explicitly indicates when generation stopped due to hitting the context window limit, rather than the requested `max_tokens` limit. This makes it easier to handle context window limits in your application logic.

```
  "usage": {
    "input_tokens": 150000,
    "output_tokens": 49950
  }
}
```

Improved tool parameter handling

Claude 4.5 models include a bug fix that preserves intentional formatting in tool call string parameters. Previously, trailing newlines in string parameters were sometimes incorrectly stripped. This fix ensures that tools requiring precise formatting (like text editors) receive parameters exactly as intended.

> This is a behind-the-scenes improvement with no API changes required. However, tools with string parameters may now receive values with trailing newlines that were previously stripped.

Example:

```
// Before: Final newline accidentally stripped
{
  "type": "tool_use",
  "id": "toolu_01A09q90qw90lq917835lq9",
  "name": "edit_todo",
  "input": {
    "file": "todo.txt",
    "contents": "1. Chop onions.\n2. ???\n3. Profit"
  }
}

// After: Trailing newline preserved as intended
{
  "type": "tool_use",
  "id": "toolu_01A09q90qw90lq917835lq9",
  "name": "edit_todo",
  "input": {
    "file": "todo.txt",
    "contents": "1. Chop onions.\n2. ???\n3. Profit\n"
  }
}
```

Token count optimizations

added tokens.

Features introduced in Claude 4

The following features were introduced in Claude 4 and are available across Claude 4 models, including Claude Sonnet 4.5 and Claude Haiku 4.5.

New refusal stop reason

Claude 4 models introduce a new `refusal` stop reason for content that the model declines to generate for safety reasons:

```
{
  "id": "msg_014XEDjypDjFzgKVWdFUXxZP",
  "type": "message",
  "role": "assistant",
  "model": "claude-sonnet-4-5",
  "content": [{"type": "text", "text": "I would be happy to assist you. You can "}],
  "stop_reason": "refusal",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 564,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 0,
    "output_tokens": 22
  }
}
```

When using Claude 4 models, you should update your application to handle `refusal` stop reasons.

Summarized thinking

With extended thinking enabled, the Messages API for Claude 4 models returns a summary of Claude's full thinking process. Summarized thinking provides the full intelligence benefits of extended thinking, while preventing misuse.

While the API is consistent across Claude 3.7 and 4 models, streaming responses for extended thinking might return in a "chunky" delivery pattern, with possible delays between streaming events.

> Summarization is processed by a different model than the one you target in your requests. The thinking model does not see the summarized output.

Interleaved thinking

Claude 4 models support interleaving tool use with extended thinking, allowing for more natural conversations where tool uses and responses can be mixed with regular messages.

> Interleaved thinking is in beta. To enable interleaved thinking, add the beta header `interleaved-thinking-2025-05-14` to your API request.

For more information, see the Extended thinking documentation.

Behavioral differences

Claude 4 models have notable behavioral changes that may affect how you structure prompts:

Communication style changes

- **More concise and direct**: Claude 4 models communicate more efficiently, with less verbose explanations
- **More natural tone**: Responses are slightly more conversational and less machine-like
- **Efficiency-focused**: May skip detailed summaries after completing actions to maintain workflow momentum (you can prompt for more detail if needed)

Instruction following

Claude 4 models are trained for precise instruction following and require more explicit direction:

- **Be explicit about actions**: Use direct language like "Make these changes" or "Implement this feature" rather than "Can you suggest changes" if you want Claude to take action
- **State desired behaviors clearly**: Claude will follow instructions precisely, so being specific about what you want helps achieve better results

For comprehensive guidance on working with these models, see Claude 4 prompt engineering best practices.

Updated text editor tool

The text editor tool has been updated for Claude 4 models with the following changes:

- **Tool type**: `text_editor_20250728`
- **Tool name**: `str_replace_based_edit_tool`

The `str_replace_editor` text editor tool remains the same for Claude Sonnet 3.7.

If you're migrating from Claude Sonnet 3.7 and using the text editor tool:

```
# Claude Sonnet 3.7
tools=[
    {
        "type": "text_editor_20250124",
        "name": "str_replace_editor"
    }
]

# Claude 4 models
tools=[
    {
        "type": "text_editor_20250728",
        "name": "str_replace_based_edit_tool"
    }
]
```

For more information, see the Text editor tool documentation.

Updated code execution tool

If you're using the code execution tool, ensure you're using the latest version `code_execution_20250825`, which adds Bash commands and file manipulation capabilities.

The legacy version `code_execution_20250522` (Python only) is still available but not recommended for new implementations.

For migration instructions, see the Code execution tool documentation.

Pricing and availability

Pricing

Claude 4.5 models maintain competitive pricing:

Model	Input	Output
Claude Opus 4.5	$5 per million tokens	$25 per million tokens

Claude Haiku 4.5	$1 per million tokens	$5 per million tokens

For more details, see the pricing documentation.

Third-party platform pricing

Starting with Claude 4.5 models (Opus 4.5, Sonnet 4.5, and Haiku 4.5), AWS Bedrock and Google Vertex AI offer two endpoint types:

- **Global endpoints**: Dynamic routing for maximum availability
- **Regional endpoints**: Guaranteed data routing through specific geographic regions with a **10% pricing premium**

This regional pricing applies to all Claude 4.5 models: Opus 4.5, Sonnet 4.5, and Haiku 4.5.

The Claude API (1P) is global by default and unaffected by this change. The Claude API is global-only (equivalent to the global endpoint offering and pricing from other providers).

For implementation details and migration guidance:

- AWS Bedrock global vs regional endpoints
- Google Vertex AI global vs regional endpoints

Availability

Claude 4.5 models are available on:

Model	Claude API	Amazon Bedrock	Google Cloud Vertex AI
Claude Opus 4.5	`claude-opus-4-5-20251101`	`anthropic.claude-opus-4-5-20251101-v1:0`	`claude-opus-4-5@20251101`
Claude Sonnet 4.5	`claude-sonnet-4-5-20250929`	`anthropic.claude-sonnet-4-5-20250929-v1:0`	`claude-sonnet-4-5@20250929`
Claude Haiku 4.5	`claude-haiku-4-5-20251001`	`anthropic.claude-haiku-4-5-20251001-v1:0`	`claude-haiku-4-5@20251001`

Also available through Claude.ai and Claude Code platforms.

Migration guide

checklists, see Migrating to Claude 4.5.

The migration guide covers the following scenarios:

- **Claude Sonnet 3.7 → Sonnet 4.5**: Complete migration path with breaking changes
- **Claude Haiku 3.5 → Haiku 4.5**: Complete migration path with breaking changes
- **Claude Sonnet 4 → Sonnet 4.5**: Quick upgrade with minimal changes
- **Claude Opus 4.1 → Sonnet 4.5**: Seamless upgrade with no breaking changes
- **Claude Opus 4.1 → Opus 4.5**: Seamless upgrade with no breaking changes
- **Claude Opus 4.5 → Sonnet 4.5**: Seamless downgrade with no breaking changes

Next steps

Best practices

Learn prompt engineering techniques for Claude 4.5 models

Model overview

Compare Claude 4.5 models with other Claude models

Migration guide

Upgrade from previous models